Lecture 7

More about classes

Lecture contents

- Destructor
- const member function
- Instance and static members
- Friend function
- Friend class
- static vs. dynamic allocated arrays using new keyword
- Memberwise assignment
- Copy constructors

Destructor

What is a destructor?

- A **destructor** is a member function that is automatically called when an object destroyed.
- It has same name as class but is preceded by a tilde (~).
- It has no arguments and return no values.
- Unlike constructors, destructors can't be overloaded.
- Destructors cannot accept arguments, so they never have a parameter list.
- Like constructors, destructors have no return type.
- Destructors are used to free memory that was dynamically allocated by the class object.

Deleting dynamic allocated objects

- If a class object has been dynamically allocated by the **new operator**, its memory should be released when the object is no longer needed.
- **sPtr** is a pointer to a dynamically allocated Sale class object. `Sale *sPtr = new Sale(1000, 0.05);`
- The **delete** operator is used to destroy the dynamically created object. `delete sPtr;`
- When the object pointed by **sPtr** is destroyed, its destructor is automatically called.

Static and dynamic allocation of objects

Only one destructor is written for a class.

```
#include <iostream>
using namespace std;
class Circle
{
private:
    double radius;
public:
   Circle(double r=1){
    radius=r;
   }
   double getRadius(){
     return radius;
   }
   void setRadius(double r){
     radius = r;
   }
   double getArea() {
     return radius*radius*3.14;
   }
   double getCircumference(){
     return 2*radius*3.14;
   }
  ~Circle(){
    cout<<"\nDestructor is invoked for radius of "<<radius;
  }
};
int main()
{
   Circle  c1;
   c1.setRadius(1);
   cout<<"Radius is : "<<c1.getRadius()<<endl;
   cout<<"Area is : "<<c1.getArea()<<endl;
   cout<<"Circumference is: "<<c1.getCircumference()<<endl;
   Circle  c2(4);
   cout<<"Radius is : "<<c2.getRadius()<<endl;
   cout<<"Area is : "<<c2.getArea()<<endl;
   cout<<"Circumference is: "<<c2.getCircumference()<<endl;
   Circle *c3=new Circle(5);
   cout<<"Radius: "<<c3->getRadius()<<endl;
   cout<<"Area: "<<c3->getArea()<<endl;
   cout<<"Circumference is: "<<c3->getCircumference()<<endl;
   return 0;
}
```

Output

```
Radius is : 1
Area is : 3.14
Circumference is6.28
Radius is : 4
Area is : 50.24
circumference is: 25.12
Radius: 5
Area: 78.5
circumference is: 31.4

Destructor is invoked for radius of 4
Destructor is invoked for radius of 1
```

- **Static objects** are deleted automatically after finishing their work, while dynamic allocated objects can't be deleted.
- The **static objects** are deleted in reverse order of construction: First constructed, last destructed.
- The dynamic allocated objects can't be deleted until you explicitly delete them using the delete operator.

```
#include <iostream>
using namespace std;
class Circle
{
private:
    double radius;
public:
   Circle(double r=1){
    radius=r;
   }
   double getRadius(){
     return radius;
   }
   void setRadius(double r){
     radius = r;
   }
   double getArea() {
     return radius*radius*3.14;
   }
   double getCircumference(){
     return 2*radius*3.14;
   }
  ~Circle(){
    cout<<"\nDestructor is invoked for radius of "<<radius;
  }
};
int main()
{
   Circle  c1;
   c1.setRadius(1);
   cout<<"Radius is : "<<c1.getRadius()<<endl;
   cout<<"Area is : "<<c1.getArea()<<endl;
   cout<<"Circumference is: "<<c1.getCircumference()<<endl;
   Circle  c2(4);
   cout<<"Radius is : "<<c2.getRadius()<<endl;
   cout<<"Area is : "<<c2.getArea()<<endl;
   cout<<"Circumference is: "<<c2.getCircumference()<<endl;
   Circle *c3=new Circle(5);
   cout<<"Radius: "<<c3->getRadius()<<endl;
   cout<<"Area: "<<c3->getArea()<<endl;
   cout<<"Circumference is: "<<c3->getCircumference()<<endl;
   delete c3;
   return 0;
}
```

Output

```
Radius is : 1
Area is : 3.14
Circumference is6.28
Radius is : 4
Area is : 50.24
circumference is: 25.12
Radius: 5
Area: 78.5
circumference is: 31.4

Destructor is invoked for radius of 5
Destructor is invoked for radius of 4
Destructor is invoked for radius of 1
```

Sale example

```
#include<iostream>
using namespace std;
class Sale
 {
  private:
    double itemCost;
    double taxRate;
  public:
    Sale(double cost=3000, double rate=0.01){
      itemCost = cost;
      taxRate = rate;
    }
    double getItemCost()  {
      return itemCost;
    }
    double getTaxRate(){
      return taxRate;
    }
    void setItemCost(double cost) {
      itemCost=cost;
    }
    ~Sale(){
      cout<<"Invoke destuctor for item: "<<itemCost<<endl;
    }
```

```
    void setTaxRate(double rate)  {
      taxRate=rate;
    }
    double getTaxAmount(){
      return itemCost * taxRate;
    }
    double getTotalCost() {
      return itemCost + (itemCost*taxRate);
    }
};
int main()
{
  Sale s3;
  cout << "The total of the sale is $"<< s3.getTotalCost()<<endl;
  Sale s4(4000);
  cout << "The total of the sale is $"<< s4.getTotalCost()<<endl;
  Sale *s1=new Sale(1000, 0.02);
  cout << "The total of the sale is $"<< s1->getTotalCost()<<endl;
  Sale *s2=new Sale(2000, 0.05);
  cout << "The total of the sale is $"<< s2->getTotalCost()<<endl;
  delete s1;
  delete s2;
}
```

```
The total of the sale is $3030
The total of the sale is $4040
The total of the sale is $1020
The total of the sale is $2100
Invoke destuctor for item: 1000
Invoke destuctor for item: 2000
Invoke destuctor for item: 4000
Invoke destuctor for item: 3000
```

```
int main() {
  Sale *s1=new Sale(1000, 0.01);
  cout << "The total of the sale is $"<< s1->getTotalCost() << endl;
  Sale *s2=new Sale(2000,0.01);
  cout << "The total of the sale is $"<< s2->getTotalCost() << endl;
  Sale *s3=new Sale(3000, 0.01);
  cout << "The total of the sale is $"<< s3->getTotalCost() << endl;
  Sale *s4=new Sale(4000,0.01);
  cout << "The total of the sale is $"<< s4->getTotalCost() << endl;
  return 0;
}
```

```
The amount of sales tax is $10
The total of the sale is $1010
The amount of sales tax is $20
The total of the sale is $2020
The amount of sales tax is $30
The total of the sale is $3030
The amount of sales tax is $40
The total of the sale is $4040
```

```
int main() {
  Sale *s1=new Sale(1000, 0.01);
  cout << "The total of the sale is $"<< s1->getTotalCost() << endl;
  Sale *s2=new Sale(2000,0.01);
  cout << "The total of the sale is $"<< s2->getTotalCost() << endl;
  Sale *s3=new Sale(3000, 0.01);
  cout << "The total of the sale is $"<< s3->getTotalCost() << endl;
  Sale *s4=new Sale(4000,0.01);
  cout << "The total of the sale is $"<< s4->getTotalCost() << endl;
  delete s1;
  delete s2;
  delete s3;
  delete s4;
  return 0;
}
```

```
The amount of sales tax is $10
The total of the sale is $1010
The amount of sales tax is $20
The total of the sale is $2020
The amount of sales tax is $30
The total of the sale is $3030
The amount of sales tax is $40
The total of the sale is $4040
Invoke destuctor for item: 1000
Invoke destuctor for item: 2000
Invoke destuctor for item: 3000
Invoke destuctor for item: 4000
```

Comments on destructors

- If you do not specify a destructor, the compiler generates a default destructor for you.

```
~circle(){}
```

- When a class contains a pointer to memory you allocate, it is your responsibility to release the memory before the class instance is destroyed.
- Classes may also only have one destructor. Because destructors take no arguments, the compiler has no way to distinguish different destructors.

Question

- In object-oriented programming, an object should protect its important data by making it public and providing a private interface to access that data. False
- **Correction**: In object-oriented programming, an object should protect its important data by making it private and providing a public interface to access that data.

Questions

- ❑ **Briefly describe the purpose of a constructor.**
- ❑ **Briefly describe the purpose of a destructor.**

- ❑ **A member function that is never declared with a return data type, but that may have arguments is**

A) The constructor
B) The destructor
C) Both the constructor and the destructor
D) Neither the constructor nor the destructor

- ❑ **A member function that is never declared with a return data type and can never have arguments is**

A) The constructor
B) The destructor
C) Both the constructor and the destructor
D) Neither the constructor nor the destructor

- ❑ **Destructor function names always start with**

A) A number
B) Tilde character (~)
C) A data type name
D) None of the above

Questions

- **A constructor that requires no arguments is called**

A) A default constructor
B) An overloaded constructor
C) A null constructor
D) None of the above

TRUE or FALSE: Constructors are never declared with a return data type.
TRUE or FALSE: Destructors are never declared with a return type.
TRUE or FALSE: Destructors may take any number of arguments.
TRUE or FALSE: A class can have more than one constructor but only one destructor.

Const member functions in C++

- **Constant member functions** are those functions that are not allowed to change the values of the data members of their class.
- It is recommended practice to make as many functions const as possible so that accidental changes to objects are avoided.

- **For function declaration within a class.**

```
return_type function_name() const;
```

```
int getName() const;
```

Const member functions in C++

- For inline function definition in .h.

```
return_type function_name() const {}
```

```
int getName() const { return name;}
```

- For function definition outside the class in .cpp.

```
return_type classname::function_name() const {}
```

```
int Student::getName() const { return name;}
```

Examples of Const Member Functions

- The program demonstrates that data members can be updated in a member function that is not constant.

11

```
#include <iostream>
using namespace std;
class Demo {
   int x;
public:
   void set_data(int a) { x = a; }
   // non const member function, data can be updated
   int get_data()
   {
       ++x;
       return x;
    }
};
int main() {
    Demo d;
    d.set_data(10);
    cout << d.get_data();
    return 0;
}
```

Examples of Const Member Functions

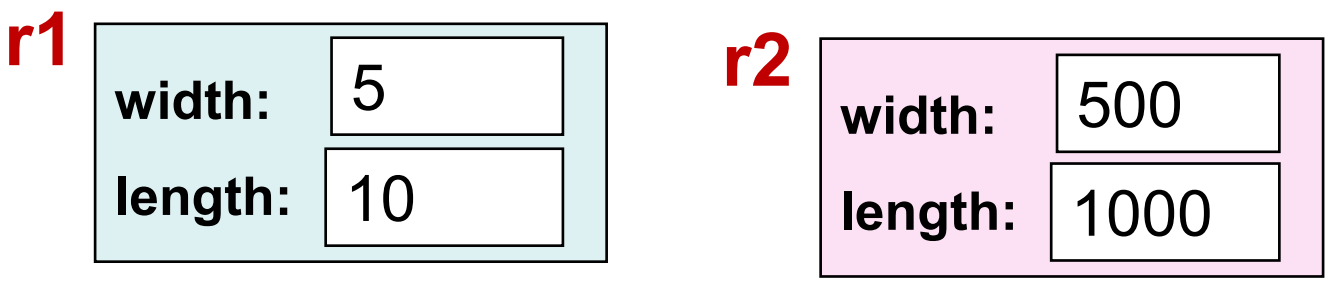
```
#include <iostream>
using namespace std;
class Demo {
   int x;
public:
   void set_data(int a) { x = a; }
   int get_data() const
   {
       ++x;  //Error: attempt to modify the data member
       return x;
    }
};
int main() {
    Demo d;
    d.set_data(10);
    cout << d.get_data();
    return 0;
}
```

- The program demonstrates that data members can't be updated in a constant member function.

```
In member function 'int Demo::get_data() const':
error: increment of member 'Demo::x' in read-only object
```

Instance variables

- They are called instance variables because each instance or object of the class has its own copies of the variables.
- `width` and `length` are called instance variables



Rectangle class

Rectangle.h

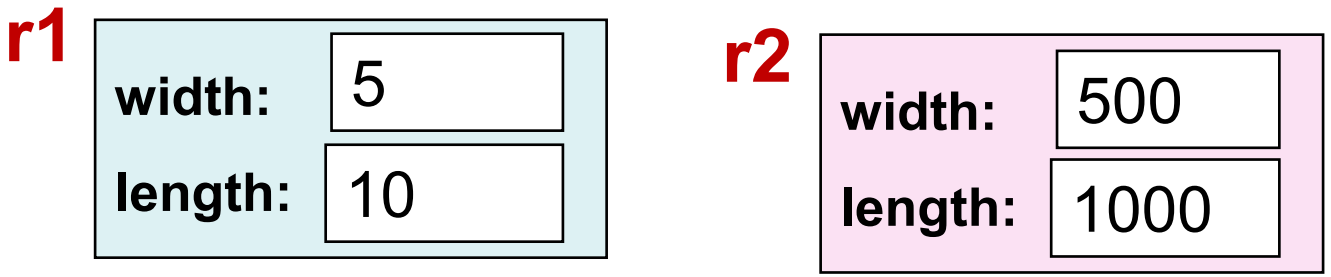
```
#ifndef RECTANGLE_H
#define RECTANGLE_H
class Rectangle
{
 private:
    double length;
    double width;
 public:
   Rectangle();
   Rectangle(double w, double l);
   double getArea();
   double getPerimeter();
};
#end if
```

Rectangle.cpp

```
#include "Rectangle.h"
Rectangle::Rectangle() {
    count_objects++;
    length=1;
    width=1;
}
Rectangle::Rectangle(double w, double l) {
    count_objects++;
    width=w;
    length=l;
}
double Rectangle::getArea() {
    return width*length;
}
double Rectangle::getPerimeter() {
    return 2*(width+length);
}
```

main.cpp

```
#include "Rectangle.h"
#include <iostream>
using namespace std;
int main(){
  Rectangle r1(5,10);
  Rectangle r2(500,1000);
  cout<<"Rectangle 1: area is "<<r1.getArea()<<" perimeter: "<<r1.getPerimeter()<<endl;
  cout<<"Rectangle 2: area is "<<r2.getArea()<<" perimeter: "<<r2.getPerimeter()<<endl;
  return 0;
}
```



Static members

- Static member variable
- Static member function

Static member variable

Static member function can be invoked using class name: A::getCount()

A.h

```
#ifndef A_H
#define A_H
class A
{
    public:
        A();
        ~A();
        static int getCount();

    private:
        static int count;
};
#endif // A_H
```

A.cpp

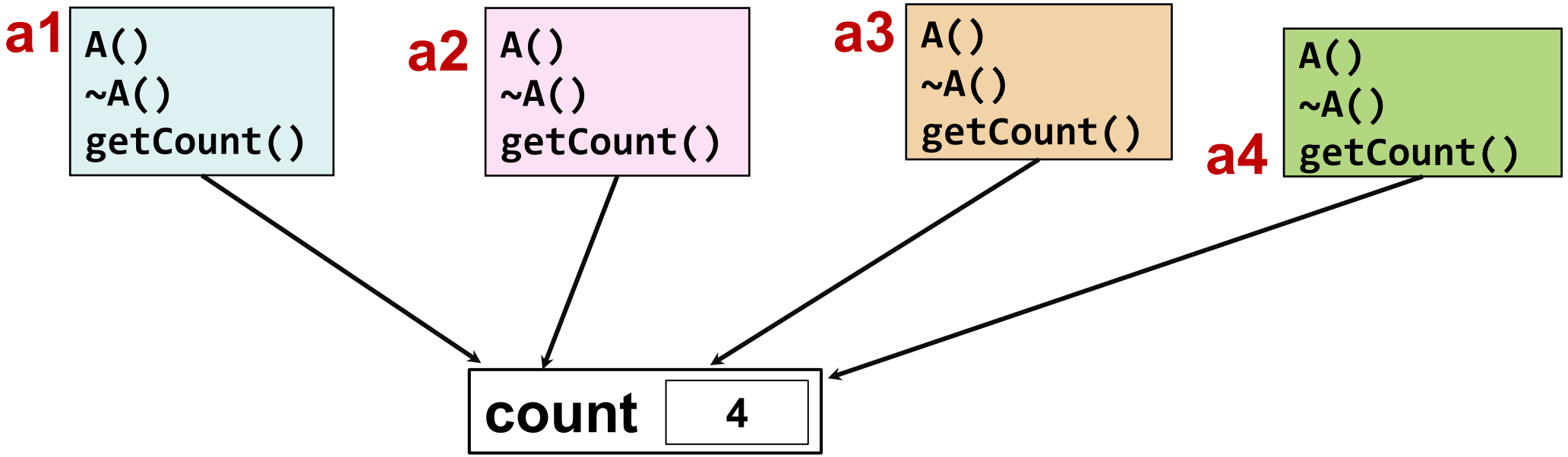
```
#include <iostream>
#include "a.h"
using namespace std;

A::A()
{
    count++;
    cout<<"Invoke constructor\n";
}

A::~A()
{
    cout<<"Invoke destructor\n";
    count--;
}
int A::getCount()
{
    return count;
}
int A::count=0;
```

main.cpp

```
#include <iostream>
#include "a.h"
using namespace std;
int main()
{
    A a1;
    A a2;
    A *a3=new A();
    A a4;
    cout<<"Number of objects: "<<a1.getCount()<<endl;
    cout<<"Number of objects: "<<a2.getCount()<<endl;
    cout<<"Number of objects: "<<a3->getCount()<<endl;
    cout<<"Number of objects: "<<a4.getCount()<<endl;
    cout<<"Number of objects: "<<A::getCount()<<endl;
    delete a3;
    cout<<"Number of objects: "<<a1.getCount()<<endl;
    cout<<"Number of objects: "<<a2.getCount()<<endl;
    cout<<"Number of objects: "<<a4.getCount()<<endl;
    cout<<"Number of objects: "<<A::getCount()<<endl;
}
```

```
Invoke constructor
Invoke constructor
Invoke constructor
Invoke constructor
Number of objects: 4
Number of objects: 4
Number of objects: 4
Number of objects: 4
Number of objects: 4
Invoke destructor
Number of objects: 3
Number of objects: 3
Number of objects: 3
Number of objects: 3
Invoke destructor
Invoke destructor
Invoke destructor
```

Static member variable



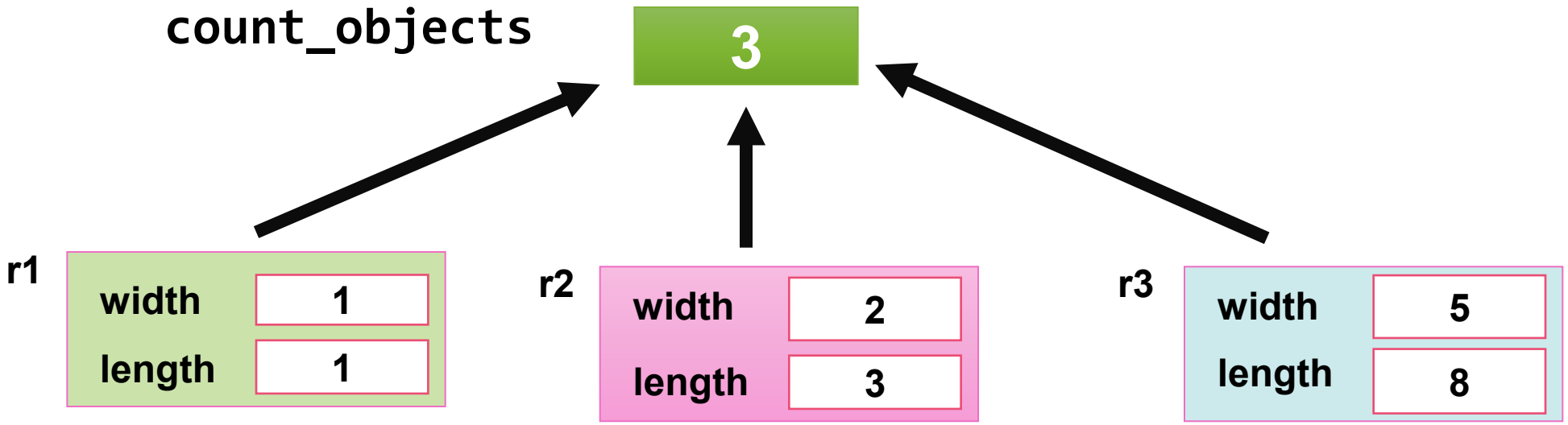
Static member variable

- When a member variable is declared with the keyword *static*, there will be only one copy of the member variable in memory, regardless of the number of instances of the class that might exist.
- static member variable is shared by all instances of the class.
- All instances of that class have access to static member variable.

Static member variable

- A class's static member variables come into existence before any instances of the class are created.
- When a value is stored in a static member variable, it is not stored in an instance of the class.

Another example for static member variable



Instances of the **Rectangle** class

Static member variable

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
class Rectangle
{
 private:
   double length;
   double width;
   static int count_objects;
 public:
   Rectangle();
   Rectangle(double w, double l);
   double getArea();
   double getPerimeter();
  int getcount_object();
};
#endif
```

```
#include "Rectangle.h"
Rectangle::Rectangle() {
   count_objects++;
   length=1;
   width=1;
}
Rectangle::Rectangle(double w, double l) {
   count_objects++;
   width=w;
   length=l;
}
double Rectangle::getArea() {
   return width*length;
}
double Rectangle::getPerimeter()  {
   return 2*(width+length);
}
int Rectangle::getcount_object () {
   return count_objects;
}
int Rectangle::count_objects=0;
```

Definition of the static member variable is written outside the class.

```
Rectangle 1: area is 1 perimeter: 4
Rectangle 2: area is 6 perimeter: 10
Rectangle 3: area is 40 perimeter: 26
Number of objects from ob1: 3
Number of objects from ob2: 3
Number of objects from ob3: 3
Number of objects from class: 3
```

```
#include "Rectangle.h"
#include <iostream>
using namespace std;
int main() {
  Rectangle r1;
  Rectangle r2(2,3);
  Rectangle r3(5,8);
  cout<<"Rectangle 1: area is "<<r1.getArea()<<" perimeter: "<<r1.getPerimeter()<<endl;
  cout<<"Rectangle 2: area is "<<r2.getArea()<<" perimeter: "<<r2.getPerimeter()<<endl;
  cout<<"Rectangle 3: area is "<<r3.getArea()<<" perimeter: "<<r3.getPerimeter()<<endl;
  cout<<"Number of objects from ob1: "<<r1.getcount_object()<<endl;
  cout<<"Number of objects from ob2: "<<r2.getcount_object()<<endl;
  cout<<"Number of objects from ob3: "<<r3.getcount_object()<<endl;
  cout<<"Number of objects from class: "<<Rectangle::getcount_object()<<endl;
  return 0;
}
```

Static member function

- If a member function is declared *static*, it may be called without any instances of the class being defined.
- static member function contains the *static* keyword in the function's prototype.

```
static returnType functionName (parameterList);
```

Static member function

- **Note:** **static member function** of a class cannot access any **non-static** member data in its class.
- Class's static member functions can be called before any instances of the class are created.
- A class's static member functions can access the class's static member variables *before* any instances of the class are defined in memory.

A.h

Static member function can be invoked using class name: A::getCount()

```
#ifndef A_H
#define A_H
class A
{
    public:
        A();
        ~A();
       static int getCount();
    private:
        static int count;
        int x=5;
};
#endif // A_H
```

ERROR: invalid use of member 'A::x' in static member function

A.cpp

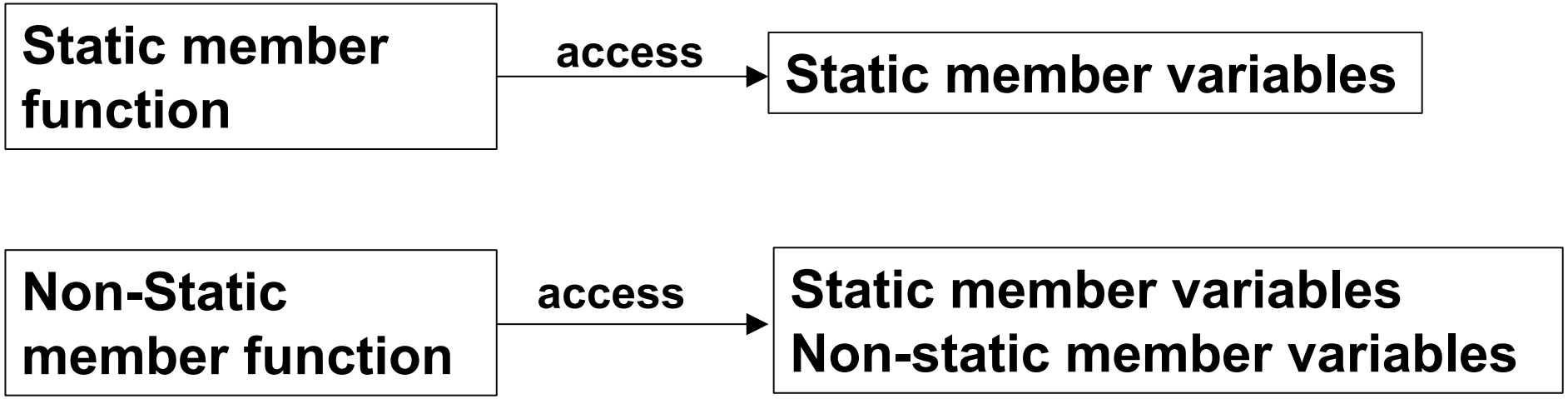
```
#include <iostream>
#include "a.h"
using namespace std;

A::A()
{
    count++;
    cout<<"Invoke constructor\n";
}

A::~A()
{
    cout<<"Invoke destructor\n";
    count--;
}
int A::getCount()
{
    //cout << x << endl;
    return count;
}
int A::count=0;
```

main.cpp

```
#include <iostream>
#include "a.h"
using namespace std;
int main()
{
    A a1;
    A a2;
    A *a3=new A();
    A a4;
    cout<<"Number of objects: "<<a1.getCount()<<endl;
    cout<<"Number of objects: "<<a2.getCount()<<endl;
    cout<<"Number of objects: "<<a3->getCount()<<endl;
    cout<<"Number of objects: "<<a4.getCount()<<endl;
    cout<<"Number of objects: "<<A::getCount()<<endl;
    delete a3;
    cout<<"Number of objects: "<<a1.getCount()<<endl;
    cout<<"Number of objects: "<<a2.getCount()<<endl;
    cout<<"Number of objects: "<<a4.getCount()<<endl;
    cout<<"Number of objects: "<<A::getCount()<<endl;

    return 0;
}
```

```
Invoke constructor
Invoke constructor
Invoke constructor
Invoke constructor
Number of objects: 4
Number of objects: 4
Number of objects: 4
Number of objects: 4
Number of objects: 4
Invoke destructor
Number of objects: 3
Number of objects: 3
Number of objects: 3
Number of objects: 3
Invoke destructor
Invoke destructor
Invoke destructor
```

Static member function



Rectangle.h

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
class Rectangle {
 private:
   double length;
   double width;
   static int count_objects;
 public:
   Rectangle();
   Rectangle(double w, double l);
   double getArea() ;
   double getPerimeter() ;
   int getcount_object() ;
   static void print() ;
};
#endif
```

print function is static

static is removed from print function in .cpp file

Rectangle.cpp

```
#include "Rectangle.h"
#include <iostream>
using namespace std;
Rectangle::Rectangle(){
  count_objects++;
  length=1;
  width=1;
}
Rectangle::Rectangle(double w, double l) {
  count_objects++;
  width=w;
  length=l;
}
double Rectangle::getArea() {
  return width*length;
}
double Rectangle::getPerimeter() {
  return 2*(width+length);
}
int Rectangle::getcount_object() {
  return count_objects;
}
void Rectangle::print() {
  cout<<"No. of objects: "<<count_objects<<endl;
}
int Rectangle::count_objects=0;//defined outside class
```

main.cpp

```
 #include "rectangle.h"
 #include <iostream>
 using namespace std;
int main()
{
  Rectangle r1;
  Rectangle r2(2,3);
  Rectangle r3(5,8);
  Rectangle::print();
  cout<<"Rectangle 1: area is "<<r1.getArea()<<" perimeter : "<<r1.getPerimeter()<<endl;
  cout<<"Rectangle 2: area is "<<r2.getArea()<<" perimeter : "<<r2.getPerimeter()<<endl;
  cout<<"Rectangle 3: area is "<<r3.getArea()<<" perimeter : "<<r3.getPerimeter()<<endl;
  cout<<"Number of objects: "<<r1.getcount_object()<<endl;
  cout<<"Number of objects: "<<r2.getcount_object()<<endl;
  cout<<"Number of objects: "<<r3.getcount_object()<<endl;
  return 0;
}
```

```
Number of objects: 3
Rectangle 1: area is 1 perimeter : 4
Rectangle 2: area is 6 perimeter : 10
Rectangle 3: area is 40 perimeter : 26
Number of objects: 3
Number of objects: 3
Number of objects: 3
```

Friend keyword

- Friend function
- Friend class

Friend of a class

- A friend is a function or class that is not a member of a class, but has access to the private members of the class.
- Private members are hidden from all parts of the program outside the class, and accessing them requires a call to a public member function.
- A *friend* function is a function that is not part of a class, but that has access to the class's private members.
- A friend function can be a regular stand-alone function, or it can be a member of another class.

Friend of a class

- In order for a function or class to become a friend of another class, it must be declared as such by the class granting it access.

```
friend ReturnType functionName (parameterList);
```

Friend function of a class

```
#include <iostream>
using namespace std;
class Distance {
  private:
     int meter;
     friend int addFive(Distance d);   // friend function
  public:
     Distance(double m=1) {
       meter=m;
     }
};
int addFive( Distance d) { // friend function definition
   d.meter = d.meter + 5; //accessing private members from the friend function
   return d.meter;
}
int main() {
   Distance d1;
   Distance d2(7);
   cout << "Distance: " << addFive(d1)<<endl;
   cout << "Distance: " << addFive(d2)<<endl;
   return 0;
}
```

```
Distance 1: 6
Distance 2: 12
```

Rectangle.h

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
class Rectangle
{
private:
    double width;
    double length;
    friend void print(Rectangle r);
public:
   Rectangle(double w=1,double l=1);
   double getWidth();
   double getLength();
   double getArea();
   double getPerimeter();
   void setWidth(double);
   void setLength(double);
};
#endif // RECTANGLE_H
```

Rectangle.cpp

```
#include<iostream>
#include "Rectangle.h"
using namespace std;
Rectangle::Rectangle(double w, double l) {
    width=w;
    length=l;
}
double Rectangle::getWidth() {
    return width;
}
double Rectangle::getLength() {
    return length;
}
double Rectangle::getArea() {
    return width*length;
}
```

```
double Rectangle::getPerimeter() {
    return 2*(width+length);
}
void Rectangle::setWidth(double w) {
    width=w;
}
void Rectangle::setLength(double l) {
    length=l;
}
void print(Rectangle r) {
    cout<<"Width: "<<r.width<<"    "<<"Length:
"<<r.length<<"   "<<"Area: "<<r.getArea()<<endl;
}
```

main.cpp

```
#include <iostream>
#include "Rectangle.h"
using namespace std;

int main()
{
    Rectangle r1;
    Rectangle r2(5,5);
    print(r1);
    print(r2);
    return 0;
}
```

```
Width: 1    Length: 1   Area: 1
Width: 5    Length: 5   Area: 25
```

14.1 What is the difference between an instance member variable and a static member variable?

14.2 Static member variables are declared inside the class declaration. Where are static member variables defined?

14.3 Does a static member variable come into existence in memory before, at the same time as, or after any instances of its class?

14.4 What limitation does a static member function have?

14.5 What action is possible with a static member function that isn't possible with an instance member function?

14.6 If class `X` declares function `f` as a friend, does function `f` become a member of class `X`?

14.7 Class `Y` is a friend of class `X`, which means the member functions of class `Y` have access to the private members of class `X`. Does the friend key word appear in class `Y`'s declaration or in class `X`'s declaration?

Friend function of a different classes

```
#include <iostream>
using namespace std;
class A {
   public:
      A() : numA(12) {} // A(){numA=12;}
   private:
      int numA;
      friend int add(); // friend function declaration
};
class B {
   public:
     B() : numB(1) {}  // B(){numB=1}
   private:
     int numB;
     friend int add(); // friend function declaration
};
int add() { //friend function accesses private members of both classes
    A objecta;
    B objectb;
    return (objecta.numA + objectb.numB);
}
int main() {
 cout << "Sum: " << add();

}
```

Sum: 13

Friend class of another class

- If class B is a friend to another class A, then class B has the right to access the private data members of class A.
- Function add in class B can access private numA in class A as class B is friend to class A.

```
#include <iostream>
using namespace std;
class B; // forward declaration of friend class as it will appear after class A
class A {
  private:
     int numA;
     friend class B;
  public:
     A():numA(12) {}
};
class B {
  private:
     int numB;
  public:
     B() : numB(1) {}
     int add() {
       A objectA;
       return objectA.numA + numB;
  }
};
int main() {
   B objectB;
   cout << "Sum: " << objectB.add();
}
```

Sum: 13

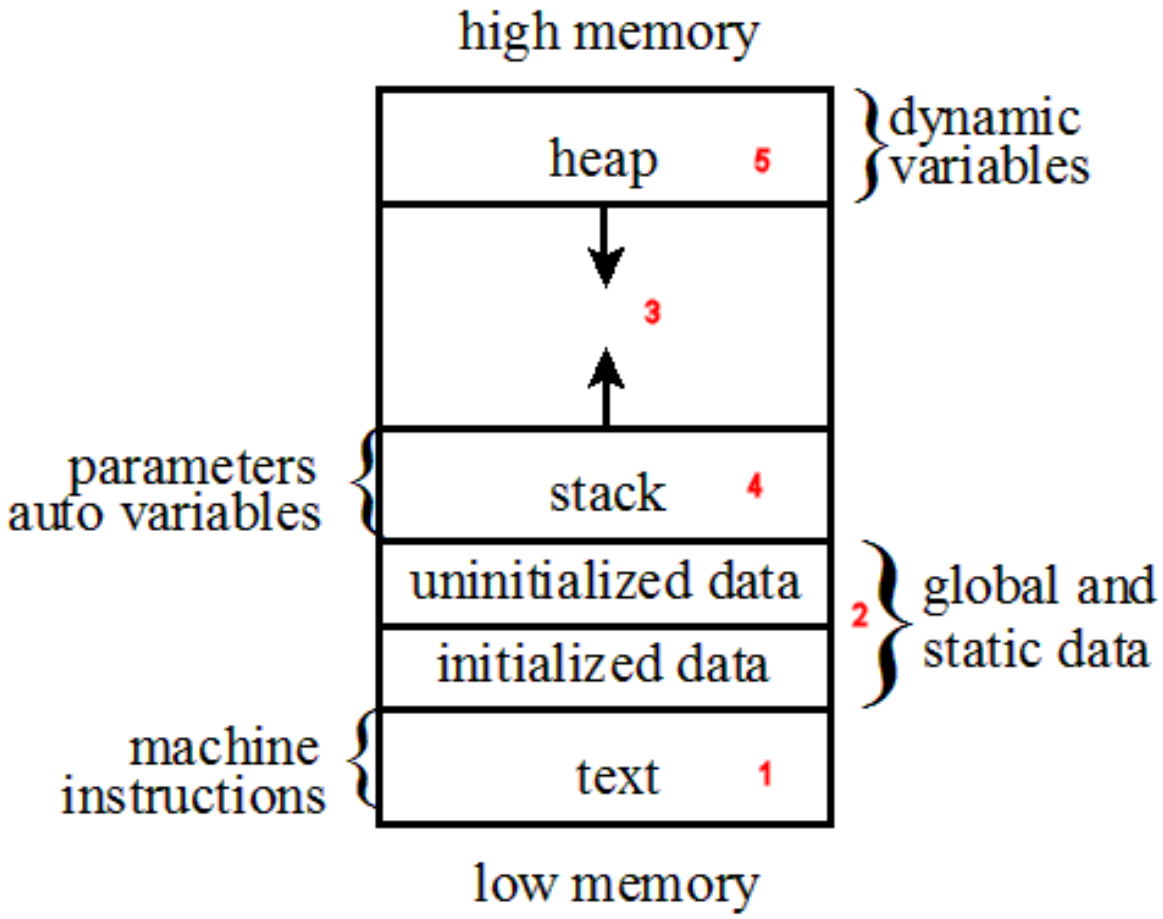
Static array vs. dynamic allocated array

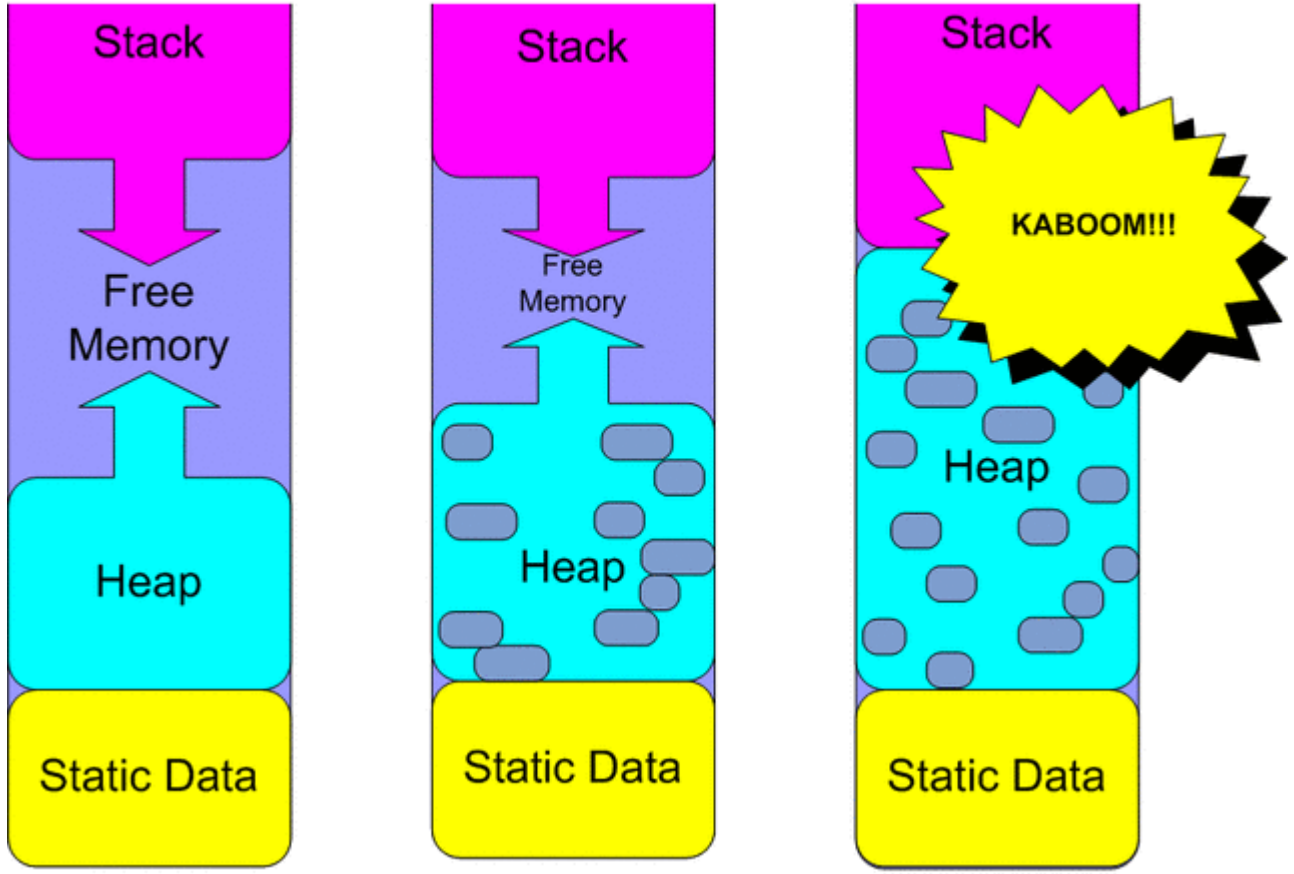
Dynamic allocated array	Static array
▪ Pointers are declared as type * var_name; `int *x=new int[5];`	▪ Arrays are declared as: type var_name [size]; `int x[5];`
▪ Array initialization `int *x = new int [5] {3,6,7,5,9};`	▪ Array initialization `int x[] = {3,6,7,5,9};`

▪ Reading array from user
`int *x=new int[5];`
`for (int i=0;i<5;i++)`
`cin >> x[i];`
▪ Printing array
`for (int i=0;i<5;i++)`
`cout<<x[i]<<endl;`
	▪ Reading array from user
	`int x[5];`
	`for (int i=0;i<5;i++)`
	`cin >> x[i];`
	▪ Printing array
	`for (int i=0;i<5;i++)`
	`cout<<x[i]<<endl;`

▪ Pointers are allocated at run-time.	▪ Arrays are allocated at compile time.
▪ `new` keyword allocates memory of the heap of the designated type and returns a pointer to it.	▪ The memory is allocated on the stack.
▪ Its size can be changed by `delete[]` to deallocate the memory and allocating a new block with a different size if desired.	▪ The size of the array is fixed once it is defined.

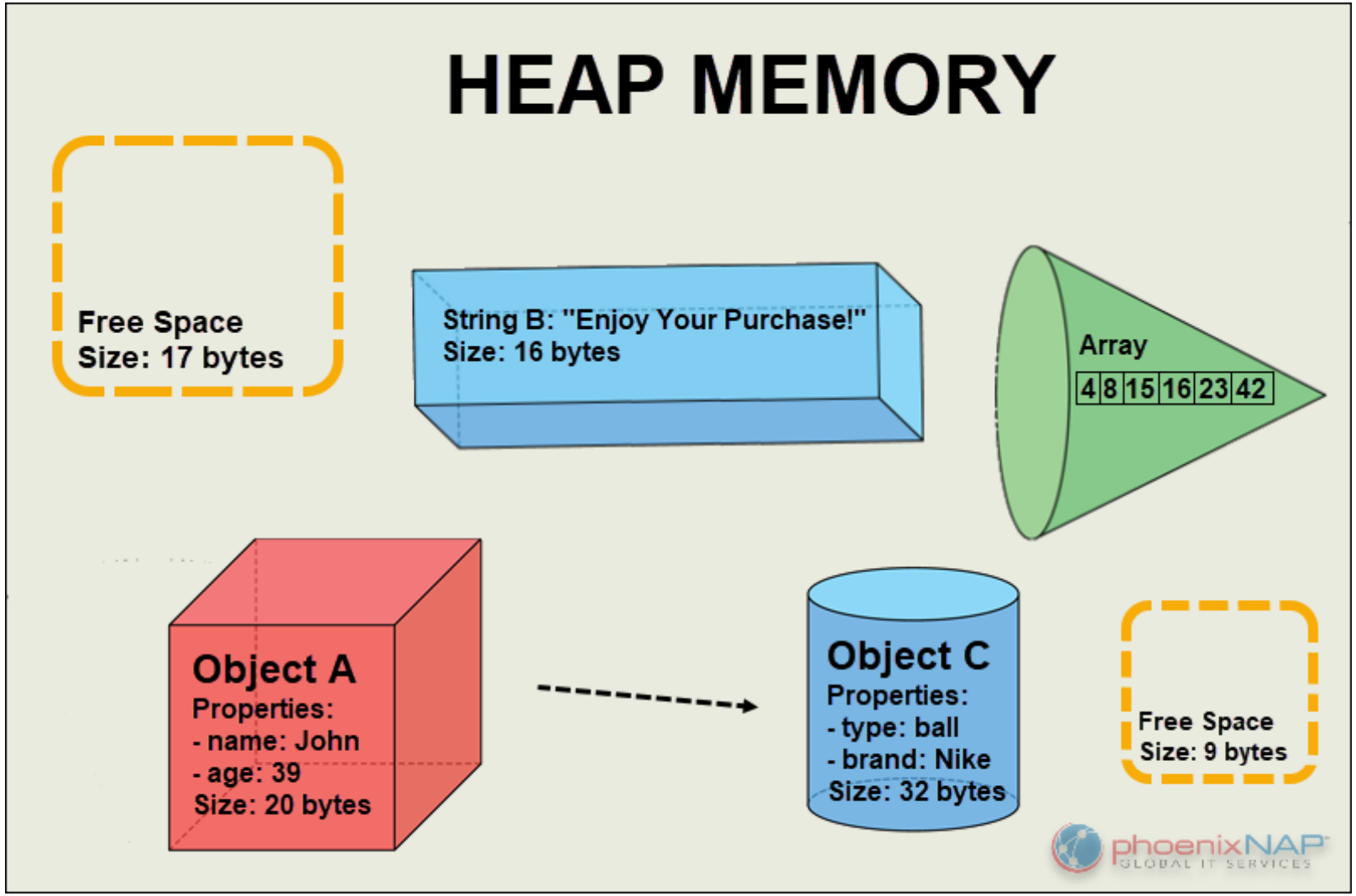
Stack vs. heap





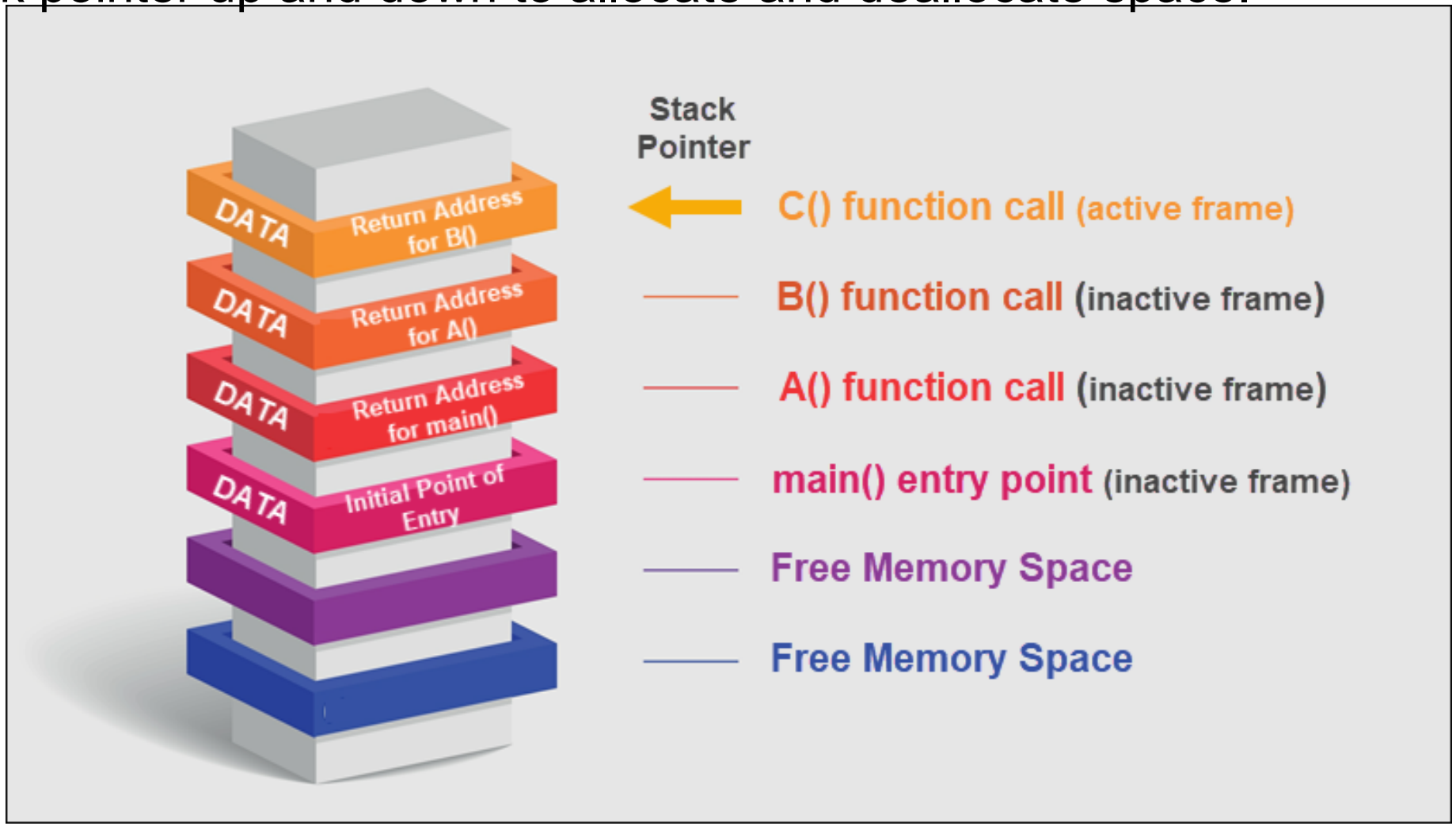
Heap

- **Large Memory Pool**. The heap provides a much larger memory pool than the stack.
- **Dynamic allocation**: Programs can allocate memory at runtime based on their needs, resulting in more efficient memory use.
- **Problem**: memory leak, slow access



stack

- Allocating and deallocating memory on the stack is fast by moving the stack pointer up and down to allocate and deallocate space.



Pointer variable

- Its content is a memory address
- There is no name associated with the pointer data type in C++.

```
int *p;  the favorite one
int* p;
int  *p;
```

Dynamic allocated array

- Its size can be changed by **delete[]** to deallocate the memory and allocating a new block with a different size if desired.

```
#include <iostream>
using namespace std;
int main()
{
    int *p = new int[3] {4, 5, 3};
    delete p;
    p = new int[4] {4,7,8,2};
    for (int i = 0; i < 4; i++)
        cout << p[i] << " ";
}
```

```
2 8 7 4
```

Employee class

```
Employee
- id: int
- name:string
- salary: double
- *children: string[]
- num_children: int
-----------------------------------------
+ Employee()
+ Employee(int, string, int, double, string*)
+ Employee(Enployee &)
+ getId():int
+ getName:string
+ setId(int):void
+ setName(string):void
+ getSalary():double
+ setSalary(double):void
+ getchild(index:int):string
+ printData():void
+ setchild(child:string,index:int):void
+ ~Employee()
+ friend print():void
```

```
return children[index];
```

```
children[index]=child;
```

```
print isn't part
of class Employee
```

Employee.h

```
#ifndef EMPLOYEE_H
#define EMPLOYEE_H
#include <string>
using namespace std;
class Employee {
private:
    int id;
    string name;
    int num_children;
    string *children;
    double salary;
public:
   Employee();
   Employee(Employee &);
   Employee(int,string,int,string*,double);
   ~Employee();
   int getId();
   void setId(int);
   string getName();
   void setName(string);
   double getSalary();
   void setSalary(double );
   double getAnnualSalary();
   void printData();
};
#endif // EMPLOYEE_H
```

Employee.cpp

```
#include<iostream>
#include "Employee.h"
using namespace std;
Employee::Employee() {
    id=1;
    name="Ali";
    num_children=3;
    children=new string[num_children] {"Ahmed","Nada","Mohamed"};
    salary=1000.0;
}
Employee::Employee(int i,string n,int num,string* child,double s){
    id=i;
    name=n;
    num_children=num;
    children=new string[num_children];
    for(int i=0; i<num_children; i++) {
        children[i]=child[i];
    }
    salary=s;
}
Employee::~Employee() {
    delete []children;
    cout<<"Deleting object of id: "<<id<<endl;
}
```

Employee.cpp

```
int Employee::getId() {
    return id;
}
void Employee::setId(int i) {
    id=i;
}
Employee::Employee(Employee &e) {
  id=e.id;
  name=e.name;
  num_children=e.num_children;
  children=new string[num_children];
  for(int i=0;i<num_children;i++)
     children[i]=e.children[i];
  salary=e.salary;
}
double Employee::getSalary() {
    return salary;
}
```

```
void Employee::setSalary(double s) {
    salary=s;
}
string Employee::getName() {
    return name;
}
void Employee::setName(string n) {
    name=n;
}
double Employee::getAnnualSalary() {
    return 12*salary;
}
void Employee::printData() {
    cout<<"Id is: "<<id<<" Name is:
"<<name<<"\nChildren: [ ";
    for(int i=0; i<num_children; i++) {
        cout<<children[i]<<" ";
    }
    cout<<"]\nSalary"<<salary<<" Annual salary:
"<<getAnnualSalary()<<endl;
}
```

main.cpp

```
#include <iostream>
#include "Employee.h"
using namespace std;
int main() {
    Employee e1;
    cout<<"e1: "<<endl;
    e1.printData();
    Employee e5 = e1; // invoke copy constructor
    cout<<"e5: "<<endl;
    e5.printData();
    Employee *e2 = new Employee();
    e2->setId(2);
    e2->setSalary(2000);
    cout<<"e2: "<<endl;
    e2->printData();
    string *children = new string[2] {"Aisel","Sally"};
    Employee e3(3,"Sausan", 2,children,3000);
    cout<<"e3: "<<endl;
    e3.printData();
    string *childrennames = new string[4];
    cout<<"Enter children names: "<<endl;
    for(int i=0; i<4; i++)
        cin>>childrennames[i];
    Employee *e4=new Employee(4,"Mohamed", 4,childrennames,3000);
    cout<<"e4: "<<endl;
    e4->printData();
    e4=e2;
    e4->printData();
    delete e2;
}
```

```
e1:
Id is: 1 Name is: Ali
Children: [ Ahmed Nada Mohamed ]
Salary1000 Annual salary: 12000
e5:
Id is: 1 Name is: Ali
Children: [ Ahmed Nada Mohamed ]
Salary1000 Annual salary: 12000
e2:
Id is: 2 Name is: Ali
Children: [ Ahmed Nada Mohamed ]
Salary2000 Annual salary: 24000
e3:
Id is: 3 Name is: Sausan
Children: [ Aisel Sally ]
Salary3000 Annual salary: 36000
Enter children names:
Tom
Jerry
James
Nasser
e4:
Id is: 4 Name is: Mohamed
Children: [ Tom Jerry James Nasser ]
Salary3000 Annual salary: 36000
Id is: 2 Name is: Ali
Children: [ Ahmed Nada Mohamed ]
Salary2000 Annual salary: 24000
Deleting object of id: 2
Deleting object of id: 3
Deleting object of id: 1
Deleting object of id: 1
```

Memberwise Assignment

- The = operator may be used to assign one object's data to another object, or to initialize one object with another object's data.
- By default, each member of one object is copied to its counterpart in the other object.
- Like other variables (except arrays), objects may be assigned to one another using the = operator.

Rectangle class

Rectangle.h

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
class Rectangle
{
 private:
    double length;
    double width;
 public:
   Rectangle();
   Rectangle(double w, double l);
   void setLength(double)
   void setWidth(double);
   double getWidth();
   double getLength();
   double getArea();
   double getPerimeter();
   void print();
};
#end if
```

Rectangle.cpp

```
#include<iostream>
#include "Rectangle.h"
using namespace std;
Rectangle::Rectangle() {
   length=1;
   width=1;
}
Rectangle::Rectangle(double w, double l) {
   width=w;
   length=l;
}
double Rectangle::getArea() {
   return width*length;
}
void Rectangle::print() {
 cout<<"width= "<<getWidth();
 cout<<" length= "<<getLength();
 cout<<" Area= "<< getArea();
 cout<<" perimeter: "<<getPerimeter()<<endl;
}
```

```
double Rectangle::getPerimeter()
{
   return 2*(width+length);
}
double Rectangle::getWidth()
{
   return width;
}
double Rectangle::getLength()
{
   return length;
}
void Rectangle::setLength(double l)
{
   length=l;
}
void Rectangle::setWidth(double w)
{
   width=w;
}
```

```
#include<iostream>
#include "Rectangle.h"
using namespace std;
int main()
{
   Rectangle r1(8,7);
   Rectangle r2(2,3);
   r1.print();
   r2.print();
   r1=r2;  //assignment operator
   cout<<"--------Values of objects after assignment operator-------"<<endl;
   r1.print();
   r2.print();
   return 0;
}
```

`r1 = r2` copies the width and length member variables of r1 directly into the width and length member variables of r2.

```
width= 8 length= 7 Area= 56 perimeter: 30
width= 2 length= 3 Area= 6 perimeter: 10
---------Values of objects after assignment operator-------
width= 2 length= 3 Area= 6 perimeter: 10
width= 2 length= 3 Area= 6 perimeter: 10
```

Assignment operator limitation

Student.h

```
#ifndef STUDENT_H
#define STUDENT_H
#include <string>
using namespace std;
class Student
{
  private:
    string name;
    double *scores;
    int numscores;
  public:
    Student();
    Student(string, int);
    ~Student();
    string getName();
    void setName(string );
    int getNumscores();
    void setNumscores(int);
    double getScore(int);
    void setScore(double, int);
};
#endif
```

Student.cpp

```
#include "Student.h"
Student::Student() {
   numscores = 3;
   scores=new
   double[numscores];
}
Student::Student(string n,int num){
   name=n;
   numscores=num;
   scores=new
   double[numscores];
}
Student::~Student() {
   delete []scores;
}
string Student::getName() {
   return name;
}
void Student::setName(string n) {
   name=n;
}
```

```
int Student::getNumscores() {
   return numscores;
}
void Student::setNumscores(int num) {
   numscores=num;
}
double Student::getScore(int index) {
   return scores[index];
}
void Student::setScore(double score,int index) {
   scores[index]=score;
}
```

```
#include<iostream>
#include "student.h"
using namespace std;
int main() {
  Student s1("Mona",6);
  for(int i=0;i<6;i++)
    s1.setScore(33,i);
  Student s2=s1; //Assignment operator
  cout<<"object 1"<<endl;
  cout << "Name: " << s1.getName() << endl;
  cout << "Test Scores: ";
  for (int i = 0; i<s1.getNumscores(); i++)
     cout << s1.getScore(i) << " ";
  cout<<endl;
  cout<<"object 2"<<endl;
  cout << "Name: " << s2.getName() << endl;
  cout << "Test Scores: ";
  for (int i = 0; i<s2.getNumscores(); i++)
     cout << s2.getScore(i) << " ";
  cout<<endl;
```

```
  s1.setScore(11,2);
  cout<<"object 1"<<endl;
  cout<<"Name: "<<s1.getName()<<endl;
  cout << "Test Scores: ";
  for(int i=0;i<s1.getNumscores();i++)
     cout << s1.getScore(i) << " ";
  cout<<endl;
  cout<<"object 2"<<endl;
  cout<<"Name: "<<s2.getName()<<endl;
  cout<<"Test Scores: ";
  for(int i=0;i<s2.getNumscores();i++)
    cout << s2.getScore(i) << " ";
  cout<<endl;
```

Problem: When doing any modification in the dynamic array of object 3, the modification happens to object 4.

```
object 1
Name: Mona
Test Scores: 33 33 33 33 33 33
object 2
Name: Mona
Test Scores: 33 33 33 33 33 33
object 1
Name: Mona
Test Scores: 33 33 11 33 33 33
object 2
Name: Mona
Test Scores: 33 33 11 33 33 33
```

Problem of assignment problem with dynamic allocation of memory

Object S1:

scores

pointer

OX356AB23F

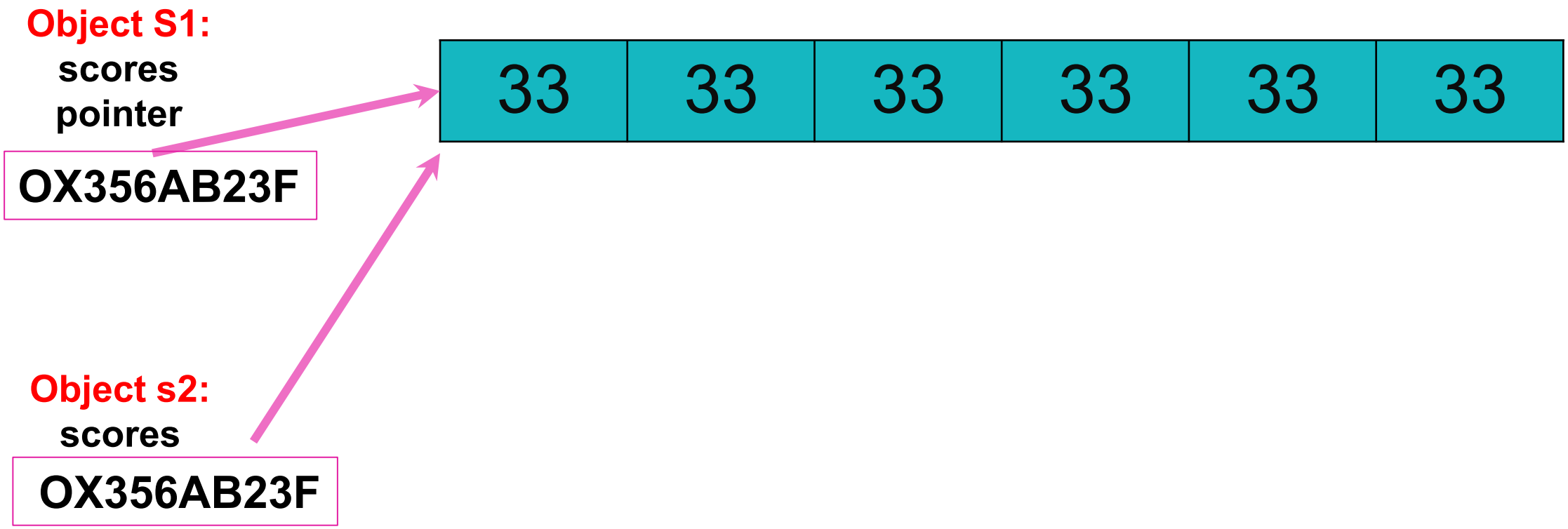
33	33	33	33	33	33

Object s2:

scores

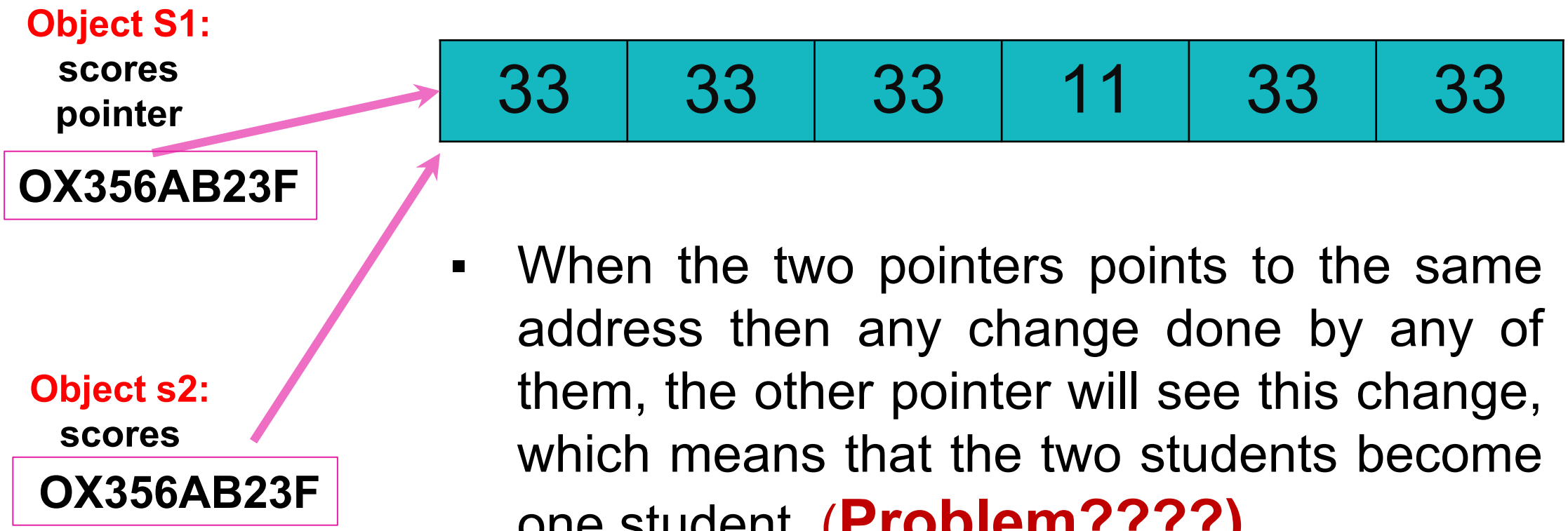
Problem of assignment problem with dynamic allocation of memory

```
S2.scores = s1.scores;
```



Problem of assignment problem with dynamic allocation of memory

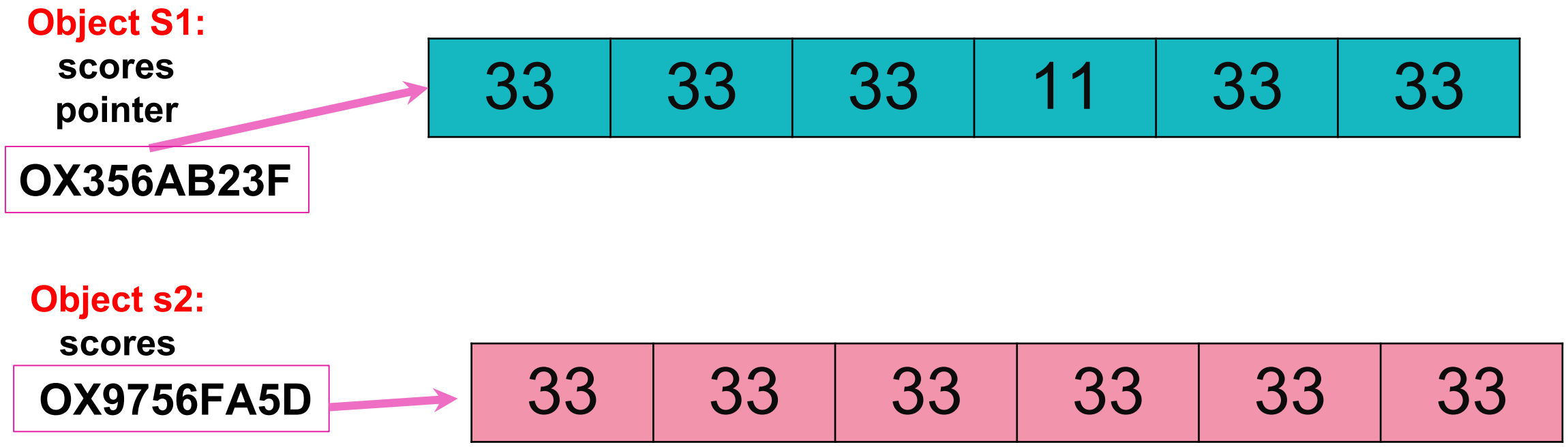
```
s1.setScore(11,2);
```



- When the two pointers points to the same address then any change done by any of them, the other pointer will see this change, which means that the two students become one student. (**Problem????)**

Problem of assignment problem with dynamic allocation of memory

```
S2.scores = s1.scores;
```



- We need that each student object has its version of scores.

Copy constructor

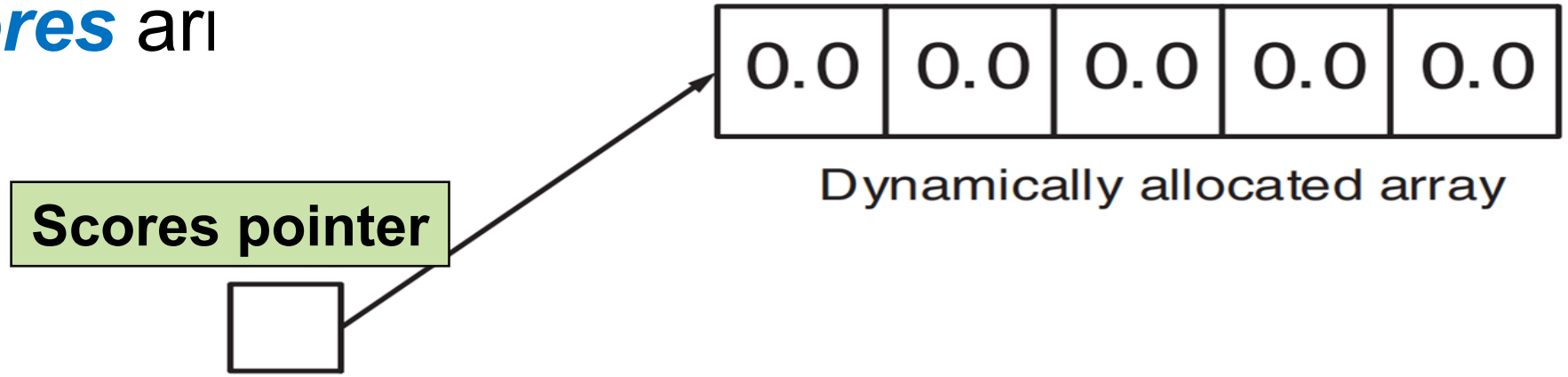
- A copy constructor is a special constructor that is called whenever a new object is created and initialized with another object's data.
- Although you may not realize it, you have seen the action of a copy constructor before. If a class doesn't have a copy constructor, C++ creates a *default copy constructor* for it.
- The default copy constructor performs the memberwise assignment.

Copy constructor

```
Student(Student &s) {  //copy constructor
   name=s.name;
   numscores=s.numscores;
   scores=new double[numscores];
   for(int i=0; i<numscores; i++)
      scores[i]=s.scores[i];
}
```

Copy constructor

- There are instances, however, where memberwise assignment cannot be used. (dynamic array).
- A potential problem with this class lies in the fact that one of its members, ***scores*** array, is a pointer.
- The pointer dynamically allocates a section of memory for the ***scores*** ar



Copy constructor (pass by reference)

- C++ requires that a copy constructor's parameter be a reference object to prevent the copy constructor from calling itself an infinite number of times
- When a class object is passed by value, the copy constructor calls itself an infinite number of times.
- This process will continue indefinitely (or at least until the available memory halts up, causing the program to halt).

Copy constructor (pass by reference)

- Copy constructor takes a reference to an object of the same class as an argument.
- If you pass the object by value in the copy constructor, it would result in a recursive call to the copy constructor itself.
- This happens because passing by value involves making a copy, and making a copy involves calling the copy constructor, leading to an infinite loop.
- Using a reference avoids this recursion.

Assignment operator limitation

Student.h

```
#ifndef STUDENT_H
#define STUDENT_H
#include <string>
using namespace std;
class Student
{
  private:
    string name;
    double *scores;
    int numscores;
  public:
    Student();
    Student(string, int);
    Student(Student &s)
    ~Student();
    string getName();
    void setName(string );
    int getNumscores();
    void setNumscores(int);
    double getScore(int);
    void setScore(double, int);
};
#endif
```

Student.cpp

```
#include "student.h"
Student::Student() {
   numscores = 3;
   scores=new double[numscores];
}
Student::Student(string n,int num){
   name=n;
   numscores=num;
   scores=new double[numscores];
}
Student::~Student() {
   delete []scores;
}
string Student::getName() {
   return name;
}
void Student::setName(string n) {
   name=n;
}
```

```
Student(Student &s) //copy constructor
{
   name=s.name;
   numscores=s.numscores;
   scores=new double[numscores];
   for(int i=0; i<numscores; i++)
      scores[i]=s.scores[i];
}
int Student::getNumscores() {
   return numscores;
}
void Student::setNumscores(int num) {
   numscores=num;
}
double Student::getScore(int index) {
   return scores[index];
}
void Student::setScore(double score,int index){
   scores[index]=score;
}
```

```
#include<iostream>
#include "student.h"
using namespace std;
int main() {
  Student s1("Mona",6);
  for(int i=0;i<6;i++)
    s1.setScore(33,i);
  // Assignment operator
  Student s2=s1; // Student s2(s1)
  cout<<"object 1"<<endl;
  cout << "Name: " << s1.getName() << endl;
  cout << "Test Scores: ";
  for (int i = 0; i<s1.getNumscores(); i++)
     cout << s1.getScore(i) << " ";
  cout<<endl;
  cout<<"object 2"<<endl;
  cout << "Name: " << s2.getName() << endl;
  cout << "Test Scores: ";
  for (int i = 0; i<s2.getNumscores(); i++)
     cout << s2.getScore(i) << " ";
  cout<<endl;
```

```
  s1.setScore(11,2);
  cout<<"object 1"<<endl;
  cout<<"Name: "<<s1.getName()<<endl;
  cout << "Test Scores: ";
  for(int i=0;i<s1.getNumscores();i++)
     cout << s1.getScore(i) << " ";
  cout<<endl;
  cout<<"object 2"<<endl;
  cout<<"Name: "<<s2.getName()<<endl;
  cout<<"Test Scores: ";
  for(int i=0;i<s2.getNumscores();i++)
    cout << s2.getScore(i) << " ";
  cout<<endl;
```

Problem: When doing any modification in the dynamic array of object 3, the modification happens to object 4.

```
object 1
Name: Mona
Test Scores: 33 33 33 33 33 33
object 2
Name: Mona
Test Scores: 33 33 33 33 33 33
object 1
Name: Mona
Test Scores: 33 33 11 33 33 33
object 2
Name: Mona
Test Scores: 33 33 33 33 33 33
```

Date.h

```
// prevent multiple inclusions of header
#ifndef DATE_H
#define DATE_H

// class Date definition
class Date
{
public:
   Date( int = 1, int = 1, int = 2000 ); // default constructor
   void print();
private:
   int month;
   int day;
   int year;
}; // end class Date

#endif
```

Date.cpp

```
#include <iostream>
#include "Date.h" // include definition of class Date from Date.h
using namespace std;

// Date constructor (should do range checking)
Date::Date( int m, int d, int y )
{
   month = m;
   day = d;
   year = y;
} // end constructor Date

// print Date in the format mm/dd/yyyy
void Date::print()
{
   cout << month << '/' << day << '/' << year;
} // end function print
```

main.cpp

```
#include <iostream>
#include "Date.h" // include definition of class Date from Date.h
using namespace std;

int main()
{
   Date date1( 7, 4, 2004 );
   Date date2; // date2 defaults to 1/1/2000

   cout << "date1 = ";
   date1.print();
   cout << "\ndate2 = ";
   date2.print();

   date2 = date1; // default memberwise assignment

   cout << "\n\nAfter default memberwise assignment, date2 = ";
   date2.print();
   cout << endl;
} // end main
```

```
date1 = 7/4/2004
date2 = 1/1/2000
After default memberwise assignment, date2 = 7/4/2004
```

Task

Student

```
- id: int
- name: string
- Numcourses: int
- degrees: double*
- coursesNames: string*
- num_Registered_Students(): static int
- getAveragedegrees():double
```

```
+ Student()
+ Student(int,string,int,string*,double*)
+ Student(Student&)
+ ~Student()
+ getId(): int
+ setId(int):void
+ getName(): string
+ setName(string):void
+ getTotalGrade():string
+ getGradeforeachdegreee(degree:double):string
+ getNumberStudents():static int
+ printData(): void
```

Each student has an **`id, name`**, a number of courses (**`numCourses`**) to study. Design a class **`Student`** with the following specifications:

- **`CoursesName`** array stores the name of studied courses.
- **`Num_Registered_students`** is a static member variable that stores the number of student objects created by the user.
- **`getAveragedegrees()`** is a **utility function** that returns the average of all degrees of each student.
- **`getId()`** and **`setId()`** are **inline functions** to return and change the id value.
- **`getTotalGrade() :`** computes the total grade for all degrees.
- **`getNumberStudents():`** is a static member function that returns number of objects.
- **`getGradeforeachdegreee(degree:double):`** returns a string array storing grade for each course.
- **`printData():`** prints data of each student **`id`**, **`name`**, **`degrees`**, **`grades`**, and **`total grade`**.

```
Id is: 1   Name is: Ali
courses: [ English Arabic Math ]
Degrees [67 89 56 ]
Grades [Good Excellent acceptable ]
Total grade: Good
Id is: 2   Name is: Mona
courses: [ a b c ]
Degrees [87 56 40 ]
Grades [Excellent acceptable fail ]
Total grade: acceptable
Id is: 3   Name is: Yasser
courses: [ z k c D ]
Degrees [37 86 40 98 ]
Grades [fail Excellent fail Excellent ]
Total grade: acceptable
No of registered students from object1: 3
No of registered students from object2: 3
No of registered student from object3: 3
No of registered students from class: 3
delete object of id: 3
delete object of id: 2
delete object of id: 1
```